How Plug-and-Play PPO Works

RL-Based Point Prompt Optimization for SAM Segmentation

Explainer for CSCE 775 Proposal 4

1. The Problem: SAM Is Sensitive to Where You Click

SAM (Segment Anything Model) can segment *any* object in an image, but you have to tell it *where* to look by giving it point prompts. The quality of the segmentation depends heavily on where those points are placed.

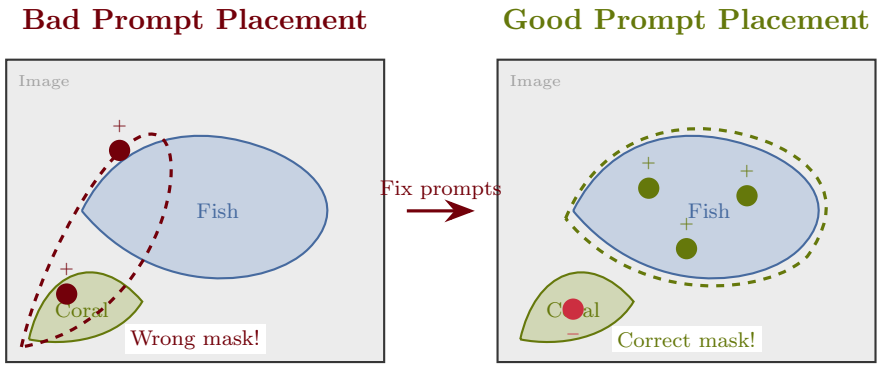

Figure 1: **The core problem.** SAM needs point prompts (+ = foreground, − = background) to know what to segment. Badly placed prompts (left) produce wrong masks. Well-placed prompts (right) produce accurate masks. The question is: how do we *automatically* find good prompt locations?

2. Step 1: Start with a Reference Image

The system begins with a *reference image* where we already know the correct segmentation (ground truth mask). It uses this to find candidate prompt locations on a new *target image*.

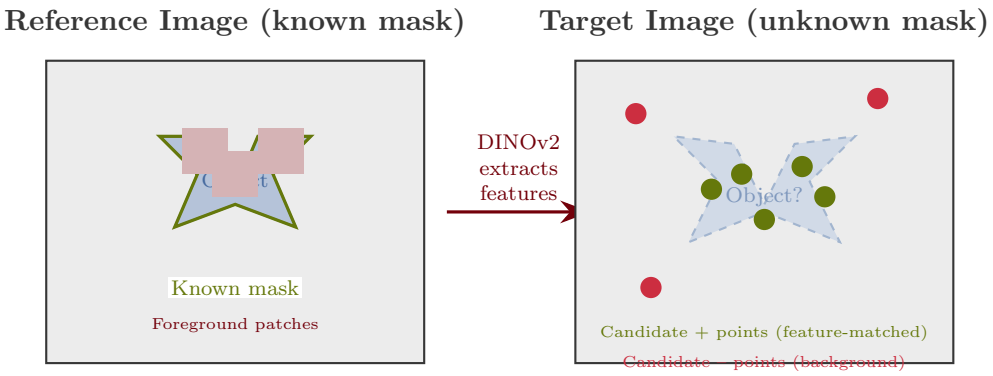

Figure 2: **Step 1: Feature matching.** DINOv2 extracts visual features from both images. Patches in the target image that are similar to foreground patches in the reference become candidate + prompts. Dissimilar patches become candidate − prompts. But we have too many candidates — which ones should we actually use?

3. Step 2: Build a Graph of Candidate Prompts

The candidate points are organized into a *dual-space graph*. Each point becomes a node, and edges encode two types of relationships: how similar the points *look* (feature space) and how far apart they *are* (physical space).

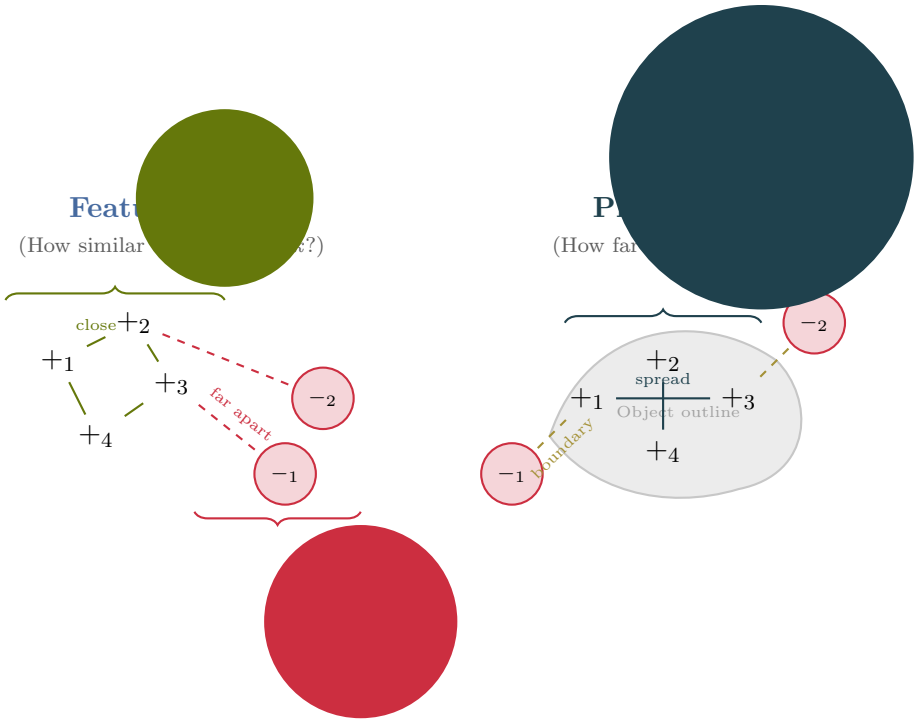

Figure 3: **Step 2: Dual-space graph.** Each candidate prompt is a node. **Feature-space** edges (left) measure visual similarity — we want + points to look alike and to look *different* from − points. **Physical-space** edges (right) measure pixel distance — we want + points spread across the object for good coverage. The RL agent will optimize this graph.

4. Step 3: The RL Agent Optimizes the Prompts

The RL agent looks at the current graph of prompts and decides how to improve it. At each step, it can add a new point, remove a bad point, or restore a previously removed point. After each action, it gets a reward based on whether the segmentation improved.

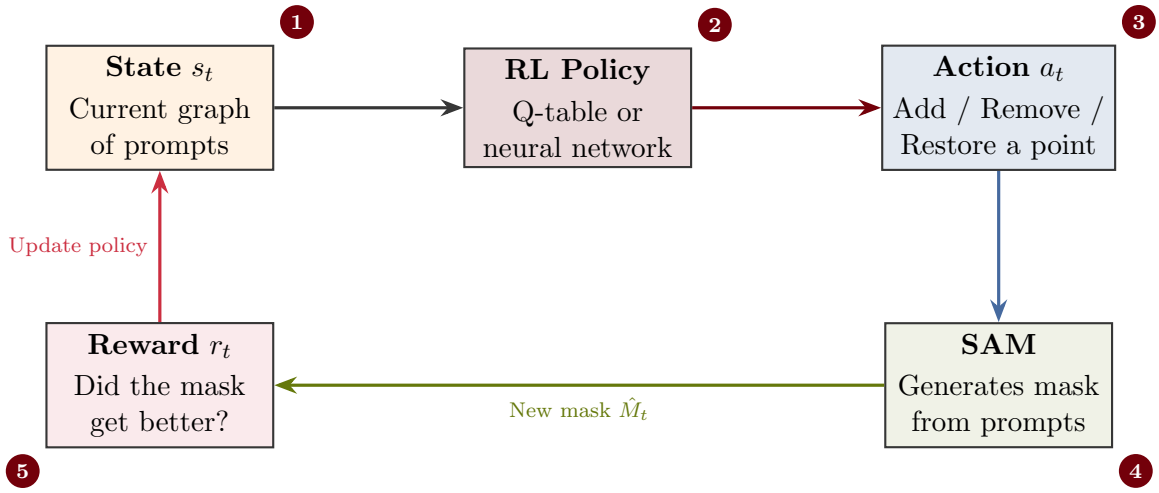

Figure 4: **Step 3: The RL optimization loop.** ① The agent observes the current prompt graph. ② The policy decides what to do. ③ It takes an action (add/remove/restore a point). ④ SAM generates a mask using the updated prompts. ⑤ The reward measures improvement and feeds back to the policy. This repeats for up to 100 steps per image.

5. The Result: Better Prompts, Better Masks

Over many iterations, the RL agent learns to remove unhelpful points and add informative ones, progressively improving the segmentation mask.

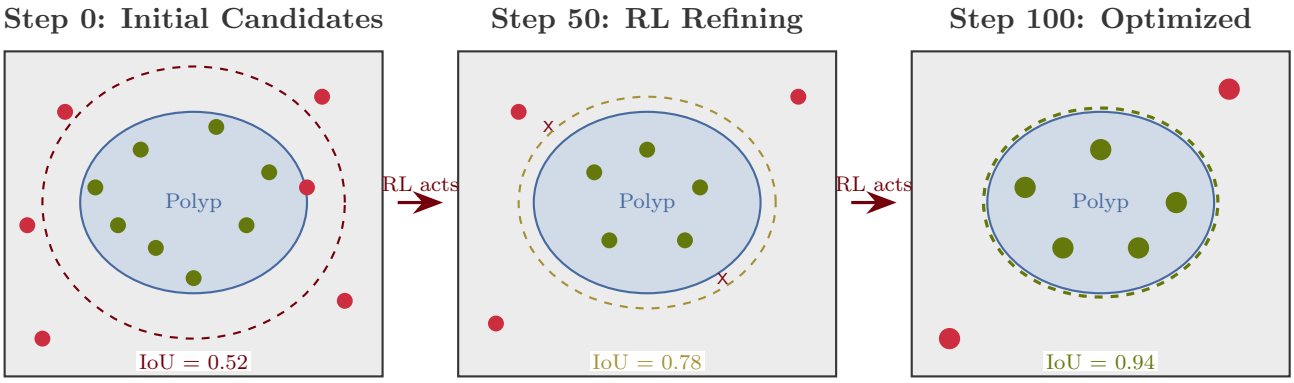

Figure 5: **Optimization over time.** The RL agent starts with many noisy candidate prompts (IoU = 0.52), iteratively removes bad ones and adds better ones (IoU = 0.78), and converges to a clean set of well-placed prompts that produce an accurate mask (IoU = 0.94). + = foreground, − = background, X = removed.

6. What We Propose to Change (Proposal 4 Extensions)

The original system uses tabular Q-learning with a proxy reward. We propose four upgrades.

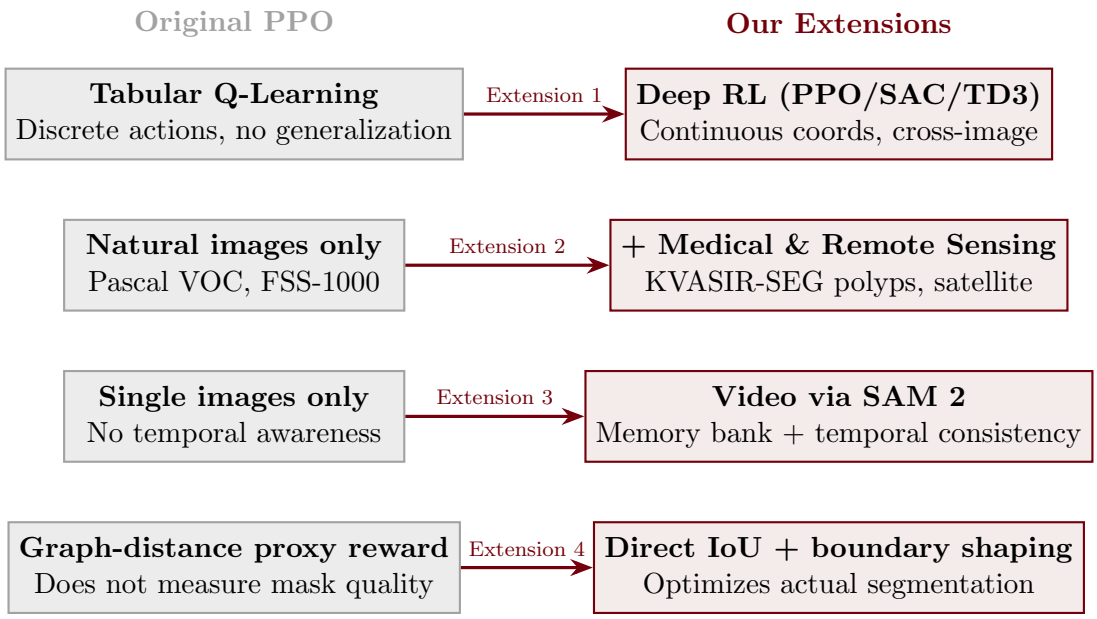

Figure 6: **Our four proposed extensions** to the Plug-and-Play PPO framework. Each replaces a limitation of the original system (left, gray) with an improved approach (right, garnet).